

PART 2: COMBINATIONAL LOGIC

GATE MINIMIZATION USING K-MAPS

SAAD BAIG – DIGITAL LOGIC DESIGN



The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from <https://www.hasanbaig.com/resources>

© Hasan Baig and Junaid Ahmed Memon. All rights reserved. These files and accompanying materials are made available under the terms of “Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) License” and is available at <https://creativecommons.org/licenses/by-nc-nd/4.0/>



GATE-LEVEL MINIMIZATION

- **Gate-level minimization** is the design task of finding an optimal gate-level implementation of the Boolean functions describing a digital circuit.
- Boolean expressions may be simplified by algebraic means but the procedure is awkward because it lacks specific rules to predict each succeeding step in the manipulative process.
- While truth tables for each function are unique, algebraic expressions are not.
- A map method known as the **Karnaugh map** or **K-map** provides a simple, straightforward procedure for minimizing Boolean functions. This method may be regarded as a pictorial form of a truth table.

TWO-VARIABLE K-MAP

A K-map is a diagram made up of squares, with each square representing one minterm of the function that is to be minimized

- Four minterms for two variables.
- Map consists of four squares, one for each minterm.

m_0	m_1
m_2	m_3

- x is primed in row 0 and unprimed in row 1.
- y is primed in column 0 and unprimed in column 1.

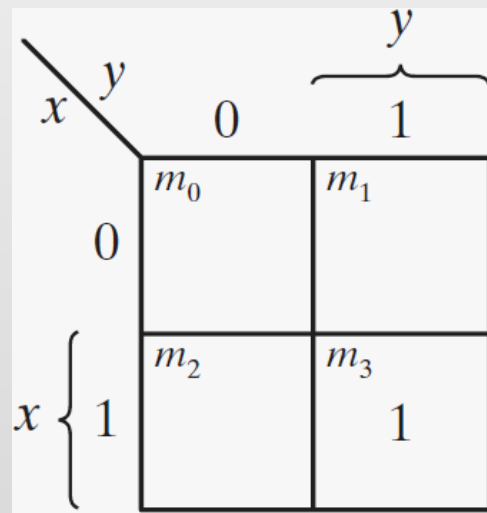
		y	
		0	1
x	0	m_0 $x'y'$	m_1 $x'y$
	1	m_2 xy'	m_3 xy

TWO-VARIABLE K-MAP

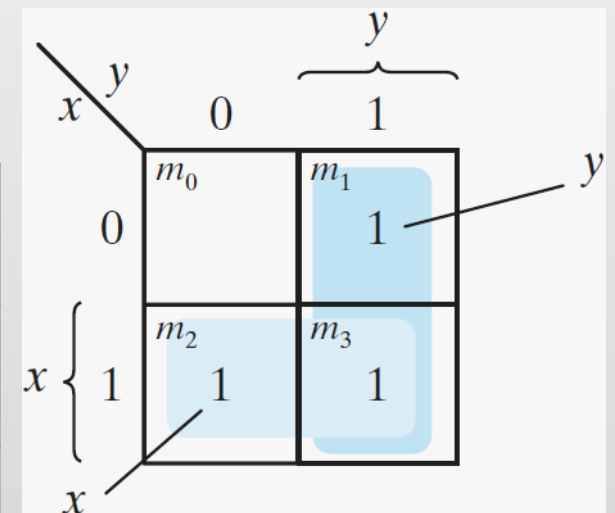
$$m_3 = x.y$$

$$m_1 + m_2 + m_3 = x'y + xy' + xy$$

x	y	x.y	Minterm
0	0	0	m_0
0	1	0	m_1
1	0	0	m_2
1	1	1	m_3



x	y	x + y	Minterm
0	0	0	m_0
0	1	1	m_1
1	0	1	m_2
1	1	1	m_3



THREE-VARIABLE K-MAP

Minterms are arranged not in a binary sequence, but in sequence similar to the Gray code. Only one bit changes in value from one adjacent column to the next.

- The square for assigned to m_5 corresponds to row 1 and column 01 (101)
- Another look to m_5 is $m_5 = xy'z$
- When variables are 0, they are primed (e.g.: x')
- When variables are 1, they are unprimed (e.g.: x)

		y			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

THREE-VARIABLE K-MAP

Any two minterms in adjacent squares (vertically or horizontally, but NOT diagonally) that are OR'ed together will cause a removal of the dissimilar variable.

- Any two adjacent squares differs by only one variable (one is primed, other is unprimed). So they can be minimized.
- Example:** $m_5 + m_7 = xy'z + xyz = xz(y' + y) = xz$

		y			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'



THREE-VARIABLE K-MAP

- **Example:** Express $F = A'C + A'B + AB'C + BC$ as (a) a sum of minterms and (b) in SOP form.

a) $F = A'B'C + AB'C + A'BC + ABC + A'BC'$

b) $F = C + A'B$

		BC			
		00	01	11	10
A	0	m ₀	m ₁ 1	m ₃ 1	m ₂ 1
	1	m ₄	m ₅ 1	m ₇ 1	m ₆

THREE-VARIABLE K-MAP

Any two minterms in adjacent squares (vertically or horizontally, but NOT diagonally) that are OR'ed together will cause a removal of the dissimilar variable.

- Some squares are adjacent, but don't touch each other.
- m_0 and m_2 are adjacent.
 - **Example:** $m_0 + m_2 = x'y'z' + x'yz' = x'z'(y' + y) = x'z'$
- m_4 and m_6 are adjacent
 - **Example:** $m_4 + m_6 = xy'z' + xyz' = xz'(y' + y) = xz'$

		y			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

THREE-VARIABLE K-MAP

- **Example:** Using K-maps, simplify the following Boolean function: $F(x, y, z) = \Sigma(2, 3, 4, 5)$

$x \backslash yz$					
		00	01	11	10
x	0	m_0	m_1	m_3 1	m_2 1
	1	m_4 1	m_5 1	m_7	m_6

- After simplification, $F(x, y, z) = x'y + xy'$

THREE-VARIABLE K-MAP

- **Example:** Using K-maps, simplify the following Boolean function: $F(x, y, z) = \Sigma(3, 4, 6, 7)$

$x \backslash yz$					
		00	01	11	10
x	0	m_0	m_1	m_3 1	m_2
	1	m_4 1	m_5	m_7 1	m_6 1

- After simplification, $F(x, y, z) = yz + xz'$

THREE-VARIABLE K-MAP

- **Example:** Using K-maps, simplify the following Boolean function: $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$

		y			
		yz	00	01	11
x	0	m_0 1	m_1	m_3	m_2 1
	1	m_4 1	m_5 1	m_7	m_6 1

- After simplification, $F(x, y, z) = z' + xy'$

FOUR-VARIABLE K-MAP

The top and bottom edges also touch each other to form adjacent squares.

- **Example:** m_3 and m_{11} are adjacent.
- One square represents one minterm, giving a term with four literals.
- Two adjacent squares represent a term with three literals.
- Four adjacent squares represent a term with two literals.
- Eight adjacent squares represent a term with one literal.
- Sixteen adjacent squares produce a function that is always equal to 1.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

		y			
		00	01	11	10
wx	yz				
	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$



FOUR-VARIABLE K-MAP

- Simplify $F = w'x'y' + x'yz' + w'xyz' + wx'y'$
- Answer: $F = x'y' + x'z' + w'yz'$

		yz			
		00	01	11	10
wx	00	m ₀ 1	m ₁ 1	m ₃	m ₂ 1
	01	m ₄	m ₅	m ₇	m ₆ 1
	11	m ₁₂	m ₁₃	m ₁₅	m ₁₄
	10	m ₈ 1	m ₉ 1	m ₁₁	m ₁₀ 1



FOUR-VARIABLE K-MAP

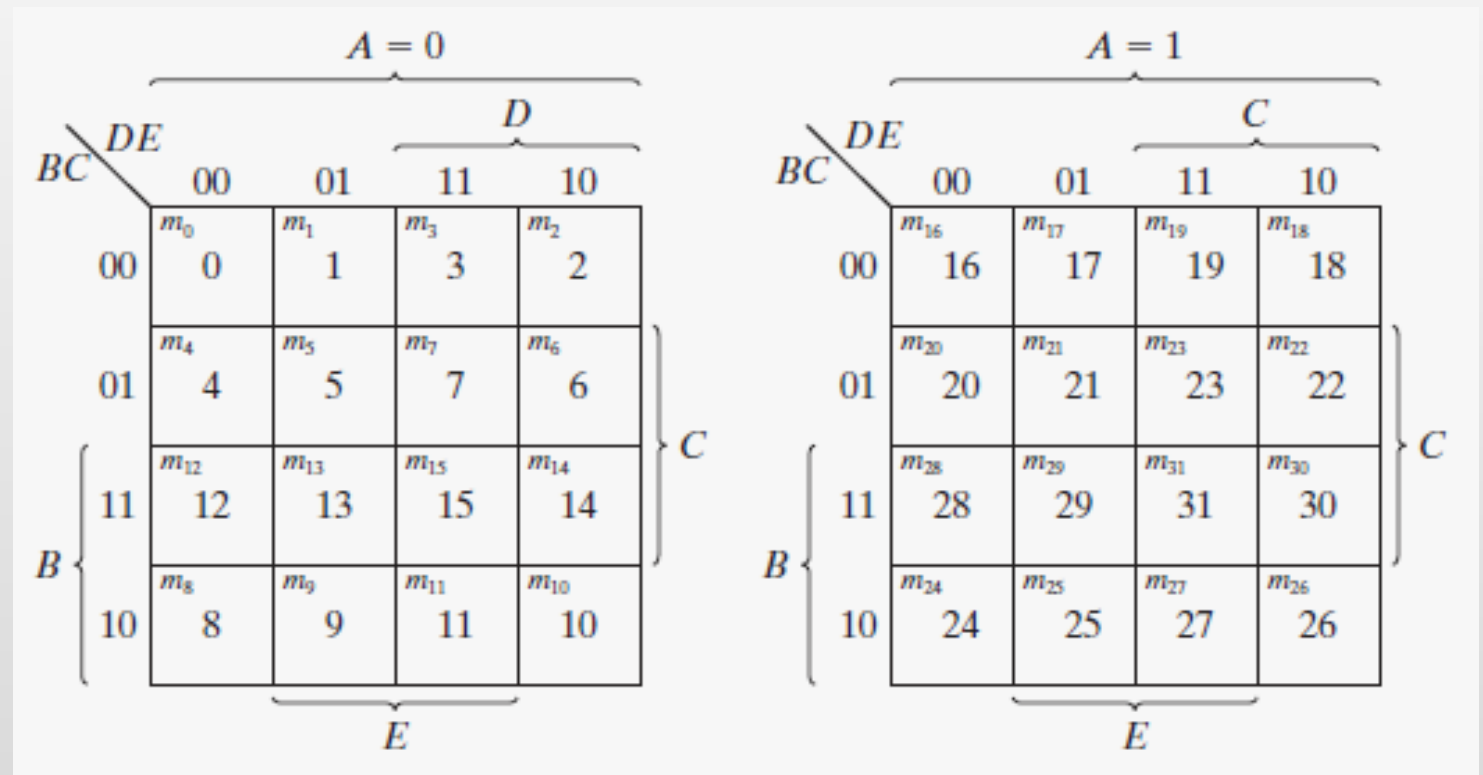
- Simplify $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$

- Answer: $F = y' + w'z' + xz'$

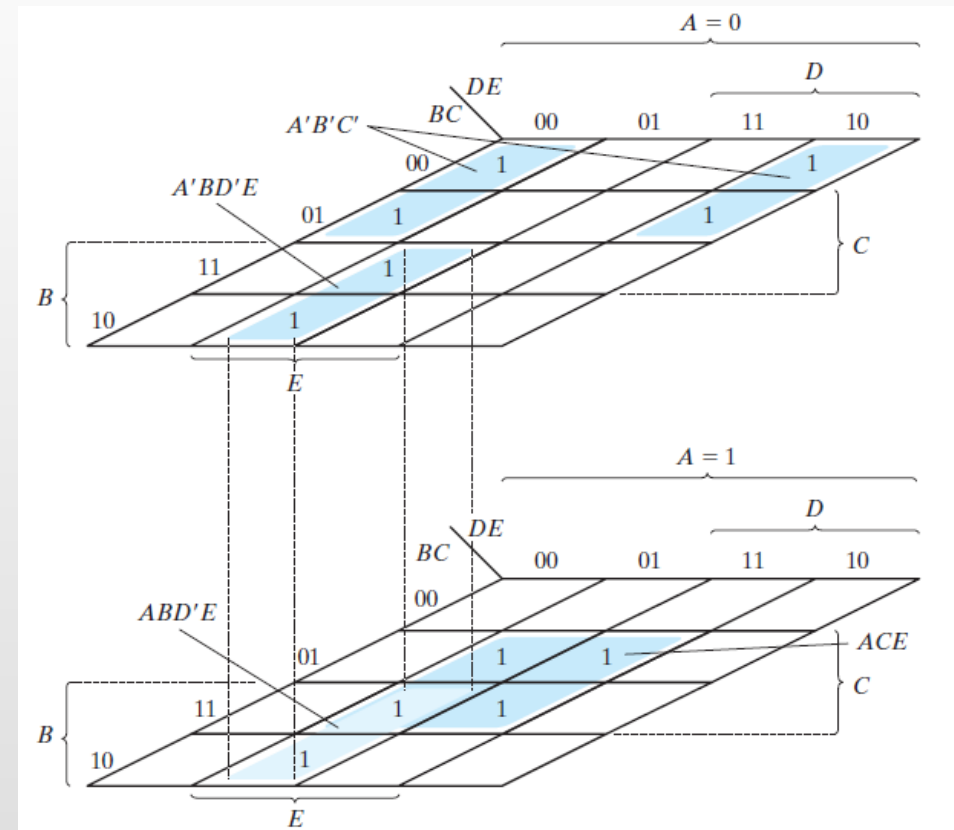
		yz			
		00	01	11	10
wx	00	m ₀ 1	m ₁ 1	m ₃	m ₂ 1
	01	m ₄ 1	m ₅ 1	m ₇	m ₆ 1
	11	m ₁₂ 1	m ₁₃ 1	m ₁₅	m ₁₄ 1
	10	m ₈ 1	m ₉ 1	m ₁₁	m ₁₀

FIVE-VARIABLE K-MAP

- 5 variables require 32 squares.
- Not usually used.



- Simplify $F(A, B, C, D, E) = \Sigma(0, 2, 4, 6, 9, 13, 21, 23, 25, 29, 31)$



- Answer: $F = A'B'E' + BD'E' + ACE$

FIVE-VARIABLE K-MAP



PRODUCT OF SUM SIMPLIFICATION

1. Mark all the minterms on K-maps with 1's as normal.
2. Mark the remaining empty squares by 0's.
3. Combine them (as we did for sum-of-products).
4. Sum-of-product expression for F' (complement of the function).
5. Using DeMorgan's Law, take the complement of F' ($(F')'$) to obtain F in the form of product-of-sum

PRODUCT OF SUM SIMPLIFICATION

- Simplify $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$ to obtain the product-of-sum expression

- Answer: $F' = AB + CD + BD'$

- $(F')' = F = (A' + B').(C' + D').(B' + D)$

		CD			
		00	01	11	10
AB	00	m ₀	m ₁	m ₃ 0	m ₂
	01	m ₄ 0	m ₅	m ₇ 0	m ₆ 0
	11	m ₁₂ 0	m ₁₃ 0	m ₁₅ 0	m ₁₄ 0
	10	m ₈	m ₉	m ₁₁ 0	m ₁₀

PRODUCT OF SUM SIMPLIFICATION

- Extract the product-of-sum expression for the following function shown in the table below.

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

x	yz			
	00	01	11	10
0	0			0
1		0	0	

$$F = (x + z).(x' + z')$$



DON'T CARE CONDITIONS

- In practice some combinations are not specified as 1's or 0's.
 - Example: Four bit binary codes has six unused combinations.
- We don't care the unspecified minterms. These minterms are called **don't care conditions**. They can be used for minimization.
- They are indicated as crosses on the map.
- They can be assumed as 1's or 0's to have best simplification.

DON'T CARE CONDITIONS

- Simplify $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$ with don't-care conditions $d(w, x, y, z) = \Sigma(0, 2, 5)$

$$F = yz + w'x'$$

m_0, m_2

wx \ yz		00	01	11	10
		m ₀	m ₁	m ₃	m ₂
00		X	1	1	X
01		m ₄	X	1	m ₆
11		m ₁₂	m ₁₃	1	m ₁₄
10		m ₈	m ₉	1	m ₁₀

$$F = yz + w'z$$

m_5

wx \ yz		00	01	11	10
		m ₀	m ₁	m ₃	m ₂
00		X	1	1	X
01		m ₄	X	1	m ₆
11		m ₁₂	m ₁₃	1	m ₁₄
10		m ₈	m ₉	1	m ₁₀



DON'T CARE CONDITIONS

- Write the product of sum expression for $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$ with don't-care conditions $d(w, x, y, z) = \Sigma(0, 2, 5)$

		yz			
		00	01	11	10
wx	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

- Answer: $F = z(w' + y)$